

Module 04

LLM Fundamentals

Bangun → Pakai → Produksikan → Adapt → Ukur



5 notebook · gratis di Colab Tesla T4 16GB · Navasena AI · 2026

Act 1

Apa & Mengapa LLM

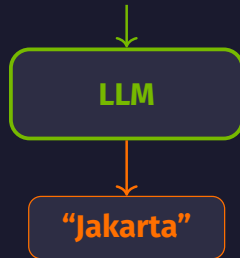
Kenapa model bahasa raksasa mengubah segalanya

Apa itu LLM?

Mesin menebak kata berikutnya yang sangat pintar

- ▶ **LLM** = *Large Language Model*: model bahasa raksasa
- ▶ Belajar dengan membaca teks sangat banyak (miliaran kata dari internet)
- ▶ Tugas intinya satu & sederhana: **menebak kata berikutnya**
- ✓ Ulangi tebakan itu terus-menerus → lahir kalimat utuh
- ▶ Dibangun di atas arsitektur **transformer** (sejak 2017)

“Ibu kota Indonesia adalah ...”

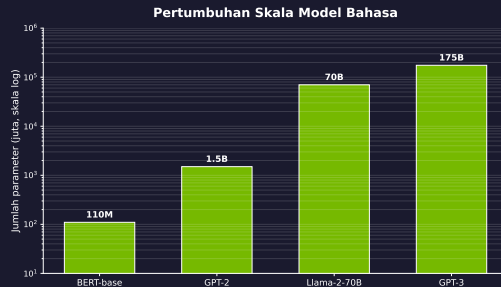


Data sangat banyak + **transformer** = tebakan kata berikutnya yang sangat jitu. Itulah “kepintaran” LLM.

Dari Membaca Satu-satu ke Membaca Sekaligus

Evolusi singkat menuju transformer

- ▶ Dulu: model membaca kata **satu per satu** (RNN/LSTM) — lambat & mudah lupa
- ▶ **Transformer** (2017): membaca **seluruh kalimat sekaligus** lewat **attention**
- ▶ Makin banyak **parameter** (“kenop”) → makin banyak pola yang bisa ditangkap
- ▶ Tapi kualitas **data** sama pentingnya dengan ukuran



Transformer memicu ledakan ukuran: dari ratusan juta jadi ratusan miliar parameter.

LLM di Kehidupan Sehari-hari

Satu model, banyak tugas bahasa

- ✓ **Chatbot & asisten:** ChatGPT, Gemini, layanan pelanggan
- ✓ **Bantu menulis kode:** membuat & menjelaskan program
- ✓ **Terjemahan:** dari satu bahasa ke bahasa lain
- ✓ **Ringkasan:** memadatkan dokumen panjang
- ✓ **Tanya-jawab:** menjawab pertanyaan informatif

Di modul ini kita akan **bangun, pakai, produksikan, adapt,** dan **ukur** LLM — semuanya gratis di Colab T4.

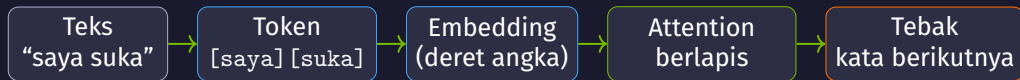
Act 2

Bagaimana LLM Bekerja

Dari teks sampai kata berikutnya

Alur Besar: Bagaimana Transformer Bekerja

Teks → token → embedding → attention berlapis → kata berikutnya



Analogi: bayangkan membaca kalimat yang kata terakhirnya ditutup, lalu kamu menebaknya. Otakmu memakai *semua* kata sebelumnya. Transformer melakukan hal yang sama — tapi dengan angka dan secara paralel. Tiap kotak di atas kita bedah di slide berikutnya.

Langkah 1 — Memecah Teks jadi Token

Komputer tidak membaca huruf; ia membaca potongan ber-nomor

- ▶ Teks dipecah jadi **token** — biasanya potongan kata
- ▶ Tiap token diberi sebuah **nomor** (ID)
- ▶ Kata umum = 1 token; kata jarang dipecah jadi beberapa
- ▶ Di notebook 00 kita pakai cara paling sederhana: **1 huruf = 1 token**



(nomor hanya ilustrasi)

Semua teks diubah jadi **token ber-nomor** dulu sebelum diolah model.

Langkah 2 — Embedding: Mengubah Makna jadi Angka

Ini adalah cara LLM “memahami” bahasa

- ▶ Tiap token diubah jadi **deret angka** (disebut *vektor*)
- ▶ Aturannya: **makna mirip** → **angka mirip**;
makna jauh → angka jauh
- ▶ “raja” dekat “ratu”; “kucing” jauh dari
“traktor”
- ▶ Jadi “memahami” di sini = menaruh tiap kata
di *peta angka* sesuai maknanya
- ▶ Model menyusun peta ini sendiri dari **pola**
dalam teks yang sangat banyak



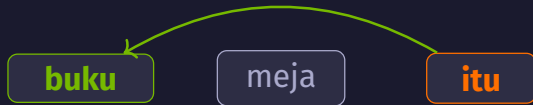
LLM “memahami” = mengenali **pola** lalu menaruh makna sebagai **angka**. Ini statistik dan pola — bukan kesadaran atau perasaan seperti manusia.

Langkah 3 — Attention: Memakai Konteks

Model belajar kata mana yang saling berhubungan

- ▶ **Attention** = model “memperhatikan” kata yang paling relevan
- ▶ Saat mengolah satu kata, ia menimbang semua kata lain di kalimat
- ▶ Contoh: model belajar “**itu**” merujuk ke “**buku**”, bukan “meja”
- ▶ Inilah cara LLM memakai **konteks** — kunci memahami kalimat panjang

“...menaruh buku di meja karena itu berat”



attention: “itu” → “buku”

Attention membuat model fokus pada kata yang benar-benar relevan, bukan menganggap semua kata sama penting.

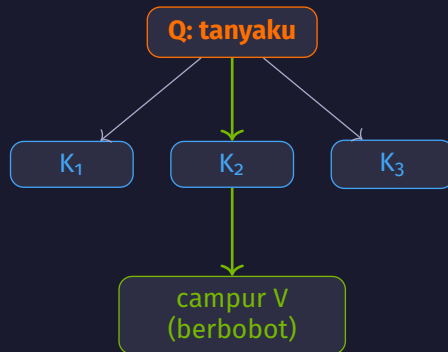
Q, K, V — Analogi Diskusi Kelompok

Mesin di dalam attention, tanpa rumus

Bayangkan satu kelas sedang diskusi. **Tiap kata adalah seorang murid:**

- ▶ **Query (Q)** = *pertanyaanku*: “murid mana yang relevan untukku?”
- ▶ **Key (K)** = *name-tag keahlian* tiap murid: “aku tahu soal X”
- ▶ **Value (V)** = *isi informasi* yang dibawa tiap murid

Cara kerjanya: cocokkan **Q**-ku ke semua **K**. Murid yang paling cocok dapat *bobot* besar, lalu aku ambil *campuran-berbobot* dari **V** mereka.



garis tebal = bobot besar (paling cocok)

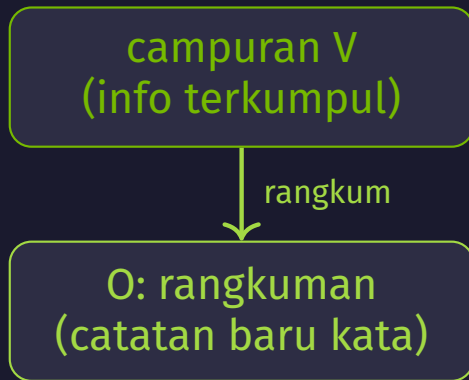
$Q = \text{pertanyaan}$ · $K = \text{label keahlian}$ · $V = \text{isi info}$. Attention = ambil campuran V yang paling cocok.

O — Merangkum Hasil Diskusi

Output projection: langkah terakhir attention

Setelah tiap kata mengambil campuran info dari kata lain, hasilnya masih “mentah”. Perlu satu langkah terakhir:

- ▶ **Output projection (O)** = merangkum info yang terkumpul jadi **catatan baru** untuk kata itu
- ▶ Analogi: setelah bertanya ke teman-teman sekelas, kamu **menulis ringkasan** di catatanmu sendiri
- ▶ Catatan baru inilah yang dipakai lapisan berikutnya

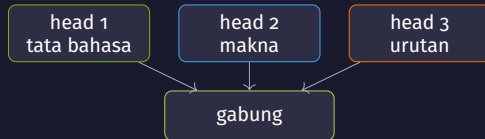


Urutan lengkap: **Q** tanya → cocokkan ke **K** → ambil campuran **V** → **O** merangkum jadi catatan baru.

Langkah 4a — Multi-Head: Banyak Diskusi Paralel

Beberapa attention sekaligus, lalu digabung

- ▶ **Multi-head** = jalankan beberapa attention sekaligus (tiap satu disebut *head*)
- ▶ Tiap head fokus pada **hubungan berbeda**: ada yang lihat tata bahasa, ada yang lihat makna, ada yang lihat urutan
- ▶ Analogi: **beberapa kelompok diskusi** berjalan paralel, lalu semua hasilnya digabung jadi satu



Banyak sudut pandang sekaligus → model menangkap lebih banyak jenis hubungan antar-kata.

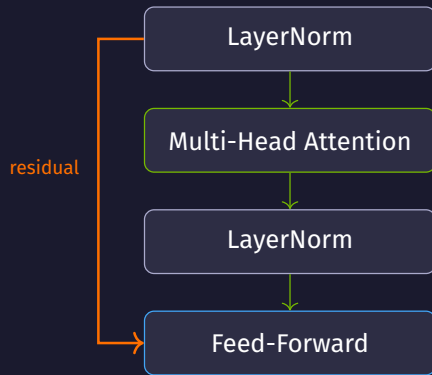
Langkah 4b — Satu Lapisan Transformer (Block)

Empat bagian yang ditumpuk berulang-ulang

Satu **transformer block** berisi empat bagian:

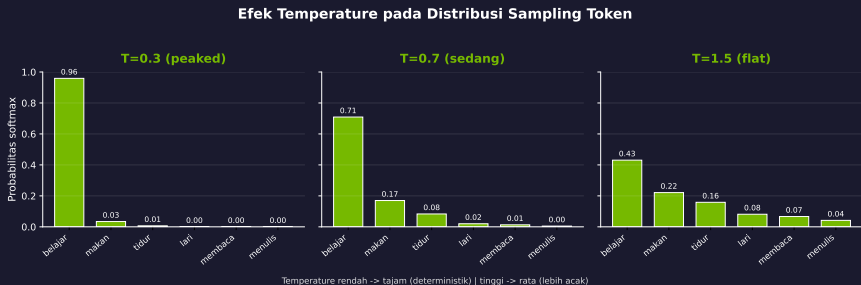
- ▶ **LayerNorm** → merapikan angka biar stabil
- ▶ **Multi-head attention** → tukar konteks antar-kata
- ▶ **Feed-forward** → mengolah tiap kata lebih dalam
- ▶ **Residual** → jalan pintas agar info awal tidak hilang

Tumpuk block ini berkali-kali → makin dalam, makin paham.



Langkah 5 — Menulis Jawaban Token demi Token

Generasi (autoregressive) & cara mengatur gayanya



- ▶ Model menebak **1 token**, lalu token itu jadi input untuk menebak token berikutnya
- ▶ Ulangi terus sampai jawaban selesai
- ▶ **temperature**: rendah = fokus/aman; tinggi = kreatif/lebih acak
- ▶ **top_p** & **repetition_penalty**: atur variasi & cegah pengulangan

Act 3

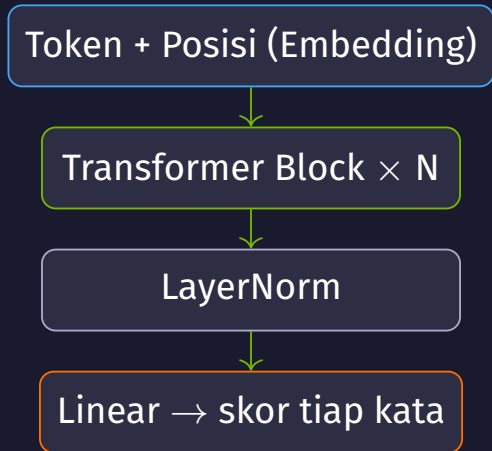
BANGUN: Transformer dari Nol

Notebook 00 — PyTorch saja, tanpa library ajaib

Membangun TinyGPT dari Nol

Menyusun sendiri tiap potongan yang baru dipelajari

- ▶ Hanya pakai **PyTorch** — tanpa library `transformers`
- ▶ **1 huruf = 1 token** (tokenizer paling sederhana)
- ▶ Susun bertahap: **self-attention** → **multi-head** → **transformer block**
- ▶ **Causal mask**: tiap token hanya boleh melihat ke kiri (masa lalu), tidak boleh mengintip masa depan
- ▶ Rakit jadi **TinyGPT** $\approx$ 0,8 juta parameter



TinyGPT $\approx$ 0,8M parameter

Self-Attention dalam Beberapa Baris

Ini adalah “diskusi kelompok” tadi, ditulis sebagai kode

```
1 # x: (batch, seq, dim) -> hitung Q, K, V lewat layer linear
2 Q, K, V = self.q(x), self.k(x), self.v(x)
3
4 scores = (Q @ K.transpose(-2, -1)) / (d ** 0.5)      # skor
      kecocokan Q vs K
5 scores = scores.masked_fill(causal_mask == 0, float('-inf')) #
      causal mask
6 weights = torch.softmax(scores, dim=-1)                # ubah jadi
      bobot (0..1)
7 out = weights @ V                                       # campuran
      berbobot dari V
```

Empat baris: cocokkan Q dengan K → tutup masa depan → jadikan bobot → campur V.

Latih & Hasilkan Teks

Korpus Indonesia kecil → model bisa melanjutkan kalimat

- ▶ Latih di **korpus Indonesia kecil**
- ▶ Angka *loss* turun → tebakan huruf makin tepat
- ▶ **Generate:** beri awalan, model lanjutkan huruf demi huruf
- ▶ Hasil belum sempurna — tapi sudah “berbau” Indonesia

```
1 for step in range(n_steps):
2     logits = model(xb)
3     loss = F.cross_entropy(
4         logits.view(-1, vocab),
5         yb.view(-1))
6     loss.backward(); opt.step();
7     opt.zero_grad()
8
9 # lanjutkan dari awalan "saya "
10 print(model.generate("saya ", n
11                       =80))
```

Dengan $\approx 0,8\text{M}$ parameter kamu sudah membangun GPT mini — konsep yang sama dipakai model raksasa.

Act 4

PAKAI: Model Instruct

Notebook 01 — Qwen2.5-3B-Instruct di Colab T4

Chat Template & System Prompt

Cara yang benar untuk “berbicara” dengan model instruct

- ▶ Model instruct mengenal format chat dengan peran: *system*, *user*, *assistant*
- ▶ **system prompt** = mengatur “kepribadian” & aturan model
- ▶ `apply_chat_template` merakit format itu otomatis
- ▶ Pakai `return_dict=True` lalu `generate(**inputs)`

```
1 msgs = [  
2     {"role": "system",  
3      "content": "Kamu asisten yang  
4         ramah."},  
5     {"role": "user", "content": "Halo  
6         !"}],  
7 ]  
8 inputs = tokenizer.  
9     apply_chat_template(  
10        msgs, add_generation_prompt=  
11            True,  
12            return_tensors="pt",  
13            return_dict=True  
14    ).to(model.device)  
15 out = model.generate(**inputs,  
16                      max_new_tokens=128)
```

Mengatur Gaya Jawaban

Parameter generasi yang sering dipakai

Parameter	Efek
temperature	Rendah = fokus/aman; tinggi = kreatif/acak
top_p	Hanya pilih dari kumpulan kata berprobabilitas terbatas
repetition_penalty	Mengurangi jawaban yang mengulang-ulang
max_new_tokens	Batas panjang jawaban yang dihasilkan

Untuk fakta: temperature rendah. Untuk ide kreatif: naikan temperature & top_p.

Prompt Engineering: Zero-shot & Few-shot

Mengarahkan model lewat cara bertanya

- ▶ **Zero-shot:** langsung minta, tanpa contoh
- ▶ **Few-shot:** beri beberapa **contoh** dulu di dalam prompt
- ▶ Tanpa melatih ulang, model meniru pola dari contoh itu
- ▶ Tunjukkan pola lewat contoh — sering lebih ampuh daripada sekadar memerintah

```
1 # few-shot di dalam prompt
2 Teks: "Filmnya bagus!"      ->
   Positif
3 Teks: "Pelayanan lambat."  ->
   Negatif
4 Teks: "Kopinya enak sekali." ->
5     <- model melanjutkan:
   Positif
```

Prompt yang baik = separuh dari hasil yang baik.

Klasifikasi Teks: Head Terlatih vs Belum

Kenapa skor bisa $\sim 0,5$ (sekadar tebakan acak)

- ▶ **bert-tiny**: bagian klasifikasi **belum** dilatih
→ label LABEL_0/1, skor $\sim 0,5$
- ▶ **distilbert-sst2**: sudah dilatih (*fine-tuned*)
→ POSITIVE/NEGATIVE yang benar
- ▶ **bart-mnli** (zero-shot): kita beri *label sendiri*, tanpa melatih

Model	Label	Skor
bert-tiny	LABEL_1	0,51
distilbert-sst2	POSITIVE	0,99
bart-mnli (zero-shot)	“ulasan bagus”	0,94

Bagian klasifikasi belum dilatih → skor $\sim 0,5$ (acak). Untuk hasil nyata: pakai model *fine-tuned* atau zero-shot.

Chatbot dengan Gradio

Antarmuka chat dalam beberapa baris kode

- ▶ **gr.ChatInterface** membuat tampilan chat siap pakai
- ▶ Fungsi kita merakit pesan lalu memanggil `generate`
- ▶ Parameter **history** disediakan Gradio otomatis
- ▶ `launch(share=True)` memberi tautan publik

```
1 def reply(message, history):
2     msgs = [{"role": "user", "
3             content": message}]
4     inputs = tokenizer.
5             apply_chat_template(
6                 msgs, add_generation_prompt
7                     =True,
8                     return_tensors="pt",
9                     return_dict=True
10                ).to(model.device)
11     out = model.generate(**inputs,
12                          max_new_tokens=128)
13     return tokenizer.decode(out[0])
14
15 gr.ChatInterface(reply).launch(
16     share=True)
```

Act 5

PRODUKSIKAN: 4-bit & Serving

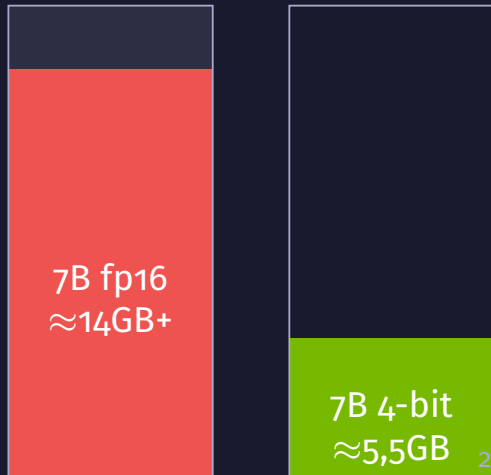
Notebook 02 — Mistral-7B muat di T4 16GB

Kenapa 7B fp16 “Meledak” di T4?

Hitungan kasar memori GPU

- ▶ Tesla T4 punya **~16GB** memori (gratis di Colab)
- ▶ **fp16** = 2 byte per parameter
- ▶ 7 miliar parameter $\times$ 2 byte $\approx$ **14GB** hanya untuk bobotnya
- ▶ Tambah data kerja lain $\rightarrow$ **OOM** (memori habis)
- ▶ Solusi: **kecilkan** bobot lewat **4-bit quantization**

T4: 16GB



4-bit NF4 Quantization

BitsAndBytesConfig — dan jebakan T4

- ▶ **NF4** = format 4-bit yang pintar untuk menyimpan bobot model
- ▶ `compute_dtype=float16` **WAJIB** di T4
- ▶ **bf16 membuat crash** di T4
- ▶ Hasil: 7B muat di **~5,5GB** memori

```
1 from transformers import
    BitsAndBytesConfig
2 import torch
3
4 bnb = BitsAndBytesConfig(
5     load_in_4bit=True,
6     bnb_4bit_quant_type="nf4",
7     bnb_4bit_compute_dtype=torch.
        float16, # bukan bf16!
8 )
9 model = AutoModelForCausalLM.
    from_pretrained(
10     "mistralai/Mistral-7B-Instruct-
        v0.3",
11     quantization_config=bnb,
        device_map="auto")
```

Profiling, Batching & Streaming

Trik agar layanan model hemat & terasa cepat

- ▶ **Profiling:** pantau pemakaian memori GPU
- ▶ **Batching:** olah banyak prompt sekaligus (set `padding_side='left'`)
- ▶ **Streaming:** tampilkan token sambil dibuat → terasa responsif

```
1 from transformers import
   TextStreamer
2
3 streamer = TextStreamer(tokenizer
4                           ,
5                           skip_prompt
6                             =True)
7
8 model.generate(**inputs,
9                max_new_tokens
10                  =200,
11                streamer=streamer)
12
13         # token
14         muncul live
15
16 tokenizer.padding_side = "left"
17         # untuk batching
```

Act 6

ADAPT: Fine-Tuning LoRA/QLoRA

Notebook 03 — ajari model hal baru tanpa GPU mahal

Masalah: Melatih Ulang Model Raksasa Itu Mahal

Miliaran “kenop” — terlalu berat untuk diubah semua

- ▶ Model raksasa punya **miliaran “kenop”** (angka bobot) yang menentukan perilakunya
- ▶ Untuk mengajarnya hal baru, kita ingin *menyetel* kenop-kenop itu
- ▶ Tapi menyetel **semua** kenop butuh GPU sangat mahal & waktu sangat lama
- ▶ Pertanyaannya: bisakah kita ajari hal baru **tanpa** menyentuh semuanya?

Jawabannya: LoRA.

Model raksasa
miliaran kenop
(mahal diubah semua)

LoRA — Tempel Sticky-Note, Jangan Tulis Ulang Buku

Bekukan model raksasa, latih catatan kecil saja

LoRA = *Low-Rank Adaptation*. Idennya:

- ▶ **Bekukan** model raksasa — jangan sentuh kenopnya sama sekali
- ▶ Tambahkan **matriks kecil** (“catatan tepi”) yang *dilatih*
- ▶ **Analogi**: daripada menulis ulang seluruh buku teks, cukup tempel **sticky-note** kecil berisi koreksi/tambahan
- ▶ Yang dilatih hanya $\sim 1\%$ dari seluruh parameter



Latih hanya sticky-note kecilnya → cepat, murah, mudah disimpan & dibagi.

QLoRA — Buku Edisi Saku 4-bit + Sticky-Note

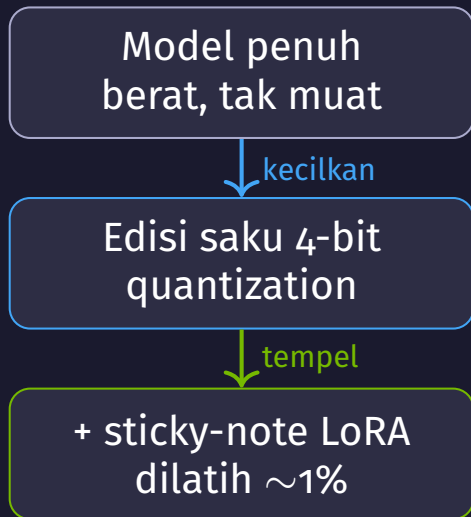
Gabungkan 4-bit dengan LoRA agar muat di T4

QLoRA = **Quantized** LoRA. Dua langkah:

1. **Kecilkan dulu** model ke **4-bit** (quantization) agar muat di GPU kecil
2. **Baru tempel** sticky-note **LoRA** di atasnya

Analogi: buku tebal diubah jadi **edisi saku 4-bit** (ringan dibawa), lalu ditemplei **sticky-note** LoRA.

- ▶ Model 4-bit tetap dibekukan; hanya sticky-note LoRA yang dilatih
- ▶ Inilah kenapa fine-tuning bisa jalan di **T4 gratis** dalam ~ 2 menit



QLoRA dalam Kode

Notebook 03 — ajari Qwen2.5-1.5B fakta-fakta Navasena

- ▶ Model 4-bit + **LoraConfig**: hanya ~1,2% parameter yang dilatih
- ▶ Dilatih lewat Trainer dalam ~2 menit
- ▶ **Bandingkan** sebelum vs sesudah: `model.disable_adapter()`
- ▶ **Simpan sticky-note-nya saja** (file kecil), bukan seluruh model

```
1 from peft import LoraConfig,
   get_peft_model
2
3 lora = LoraConfig(
4     r=8, lora_alpha=16,
5     lora_dropout=0.05,
6     target_modules=["q_proj", "
7                     v_proj"],
8     task_type="CAUSAL_LM",
9 )
10 model = get_peft_model(base_4bit,
11                        lora)
12 model.print_trainable_parameters()
13     # ~1.2%
14
15 with model.disable_adapter():
16     # ... model.eval()
```

Act 7

UKUR: Evaluasi

Notebook 04 — model mana yang benar-benar lebih baik?

Metrik Otomatis

Mengukur kualitas jawaban dengan angka

Metrik	Mengukur apa
ROUGE	Tumpang-tindih kata dengan jawaban acuan (untuk ringkasan)
BLEU	Kecocokan frasa (sering untuk terjemahan)
BERTScore	Kemiripan <i>makna</i> , bukan kata yang persis sama
Perplexity	Seberapa “terkejut” model — makin rendah makin lancar

ROUGE/BLEU = cocok-kata; BERTScore = cocok-makna; Perplexity = kelancaran model.

Apakah Bedanya Nyata?

Skor lebih tinggi belum tentu benar-benar lebih baik

- ▶ Model B skornya sedikit lebih tinggi — tapi apa itu cuma **kebetulan**?
- ▶ Kita uji dengan `p-value` (paired t-test)
- ▶ $p < 0,05 \rightarrow$ beda **nyata** (kecil kemungkinan kebetulan)
- ▶ $p > 0,05 \rightarrow$ **belum** bisa disimpulkan lebih baik

Hasil	Arti
$p = 0,12$	belum nyata
$p = 0,01$	nyata

Pelajaran: metrik boleh memihak model yang lebih besar, tapi $p > 0,05$ berarti bedanya belum meyakinkan.

LLM-as-Judge: Lokal & Cloud

Pakai LLM lain sebagai “juri” untuk menilai jawaban

- ▶ Beri jawaban A & B ke LLM “juri”, minta ia menilai mana yang lebih baik
- ▶ **Lokal**: pakai Qwen3B yang sudah kita muat (gratis)
- ▶ **Cloud (opsional)**: NVIDIA NIM di build.nvidia.com → model lebih besar
- ▶ Akses API cloud kini **diizinkan** untuk model raksasa

```
1 from openai import OpenAI
2
3 client = OpenAI(
4     base_url="https://integrate.
5         api.nvidia.com/v1",
6     api_key=NVIDIA_API_KEY)
7
8 judge = client.chat.completions.
9     create(
10         model="meta/llama-3.1-70b-
11             instruct",
12         messages=[{"role": "user",
13                     "content": f"Nilai
14                         jawaban: {ans}
15                     "}]])
```

Act 8

Ringkasan & Lanjutan

Lima notebook, satu perjalanan

Ringkasan: 5 Notebook Modul 04

#	Tahap	Model	Yang dipelajari
00	BANGUN	TinyGPT (~0,8M)	Attention, causal mask, multi-head, block — rakit dari nol
01	PAKAI	Qwen2.5-3B-Instruct	Chat template, parameter generasi, prompt eng., klasifikasi
02	PRODUKSIKAN	Mistral-7B-Instruct	4-bit NF4, profiling, batching, streaming
03	ADAPT	Qwen2.5-1.5B	QLoRA: 4-bit + sticky-note, latih ~1%, ~2 menit
04	UKUR	Qwen3B + NIM cloud	ROUGE/BLEU/BERTScore, p-value, LLM-as-judge

Semua gratis di Colab **Tesla T4 16GB**, semua model **non-gated**.

Setup T4 & Lanjut ke Module 05

Aturan main agar tidak crash — dan kenapa kita butuh RAG

Aturan aman di T4:

- ▶ Pin transformers>=4.44,<5
- ▶ 4-bit compute_dtype=float16 (**bf16 crash**)
- ▶ apply_chat_template(..., return_dict=True) lalu generate(**inputs)
- ▶ Cek GPU dengan !nvidia-smi

```
1 !pip install "transformers  
    >=4.44,<5" \  
2             bitsandbytes  
               accelerate  
               peft  
3 !nvidia-smi
```

LLM bisa **mengarang** (halusinasi) & punya **batas pengetahuan** → **Module 05 (RAG)** memberi LLM “buku contekan”.

Terima Kasih!

Pertanyaan? Diskusi?

LLM Fundamentals · Modul 04

Bangun → Pakai → Produksikan → Adapt → Ukur
Navasena Gen-ML Course